CW EXPENSE TRACKER

1. **Overview:** Design an application called expense tracker, in which a user can list and track down his/her every single expense.
2. **Features:**
    a. User Registration
    b. User Login
    c. Category Management (Food, Travelling, Recharge, etc.)
    d. Budget Allocation (in this user will enter his monthly income amount)
    e. Expense Management
    f. User Dashboard (in this user can see total incurred monthly expense, amount of money left, expense incurred by category)
3. **Tech Stack:**
    a. For frontend:
        i. React.js
        ii. MUI
        iii. Material React Table npm library for table related ui
        iv. Axios (for api calling)
        v. Zod (for user input validations)
        vi. Redux Toolkit (for local state management)
        vii. MUI date picker libraries
        viii. Day.js
        ix. React router dom (for managing pages)
        x. React PDF
        xi. PDF.js Dist
        xii. Prop Types
    b. For api gateway:
        i. Spring boot api gateway
    c. For backend
        i. Node.js
        ii. CORS
        iii. Helmet (for security headers)
        iv. Joi (for data validations)
        v. JWT (for maintaining user login sessions)
        vi. Typescript
        vii. Multer (for file uploads)
        viii. Express (for creating api server)
        ix. Day.js
        x. Bcrypt.js (for password hashing)
    d. For backend for frontend
        i. Node.js
        ii. Axios (used at backend service)
        iii. Joi (for input validations)
        iv. Day.js
    e. For caching
        i. Redis
    f. For backend logs

    i. Winston with log rotation (for node bff, node backend services)
    ii. Slf4j (for spring boot api gateway)
g. For database
    i. PostgreSQL
h. For ORM
    i. Sequelize
i. For encryption and decryption
    i. AES 256 bit
j. For linting
    i. ESLint

4. **DTOs:**
    a. **User Registration**
        i. Name (Required, max 32 characters, except space no special character is required, use MUI text field of type text for this)
        ii. Email (Required, max 32 characters, rfc validations, use MUI text field of type email for this)
        iii. Phone No (optional, max 10 characters, only Indian numbers are allowed, use MUI text field of type tel for this)
        iv. Password (Required, max 25 characters and minimum 8 characters, one small, one capital, one special symbol is required, use MUI text field of type password for this)
        v. Employment Type (Required, MUI drop-down, Salaried, Self-Employed)
        vi. Captcha (Required, max 8 characters, use MUI text field for this)
        vii. Profile Photo (Optional, max 1MB is allowed, only *.png, *.jpeg, *.jpg files are allowed, please use FileUploader predefined react component for this)
    b. **User Login**
        i. Email Address (Required)
        ii. Password (Required)
        iii. Captcha (Required)
    c. **Category Management**
        i. Category Name (Required, max 100 characters, letters, numbers, special characters like dash, underscore, space, forward slash are allowed)
        ii. Active flag (Boolean type, will be maintained by the backend, user can only toggle it)
        iii. Delete flag (Boolean type, will be maintained by the backend, will set to false when user will delete any category)
        iv. Captcha Text (Required, use MUI text field of type text for this)
    d. **Expense Management**
        i. Expense Name (Required, max 64 characters, only letters and numbers are allowed, use MUI text field of type text for this)
        ii. Expense Amount (Required, only numbers are allowed but not scientific notation, use MUI text field of type number for this)

iii. Expense Category (Optional, by default will set to not defined option, its option will come from categories master, use MUI auto complete for this)
iv. Expense Type (Required, use MUI select for this, possible options are Fixed and Variable)
v. Expense Date (Optional, use MUI date picker library for this, use YYYY-MM-DD format throughout all services)
vi. Captcha Text (Required, use MUI text field of type text for this)

e. Budget Allocation
   i. Budget Amount (Required, use MUI text field of type number but not any scientific notation for this)
   ii. From Date (Required, MUI date picker component)
   iii. To Date (Required MUI date picker component)
   iv. Captcha Text (Required, use MUI text field of type text for this)
f. User Dashboard
   i. Please decide yourself that how to show and process dashboard data according to the requirements

5. **External Services**
   a. Please use captcha service running at http://localhost:11905 for all of the captcha needs.
   b. Please use Redis running at localhost 6379 port for all of the Redis related tasks.
   c. Please use postgres database running at localhost port number 5432 for all of the database tasks. Please use the following credentials while connecting with the database
      i. Username: postgres
      ii. Password: root
      iii. Database name: cw-expense-tracker
6. **Application and Service Design Rules**
   a. Please prefer to create a separate plug and play components in frontend if you think if it will be used in many places such as Captcha, Sidebar, etc.
   b. Please use latest node LTS version throughout all MERN stack-based services.
   c. In frontend for every component, keep the logical function in its separate hooks
   d. If any component is being used at many places then create that component under components/core folder and its hook under hooks/core folder.
   e. In backend keep separate folders for every separate concern such as controllers will be in controllers folder, routes will be in routes folder, middlewares will be in middlewares folder, etc.
   f. To reduce the headache of preparing builds for small things, please define global environment variables for things like max file upload size, etc. This is valid for all the services

g. Frontend will send its request to the bff service and from there request will arrive at api gateway and from there normal flow will continue.
h. Please use secure http only cookies for maintaining login related tokens such as access token, etc.
i. Please use ES Lint for linting of all services.
j. Please do not create test cases for any of the backend services as of now but while designing the backend, please design it in such a way that its test cases can be written with an ease in future.
k. Please follow DRY and SOLID principles throughout all the services (frontend, backend, etc.)
l. Please do not define all the models in the same public schema, instead please create a separation of concerns. For instance, put all the masters related models in the master schema.
m. Please create configuration files for services like Redis, Databases, Logs, etc. in config folders. Also, please follow Singleton Design Patterns for this config classes so that only one object is created for these classes throughout the application.
n. Please avoid OWASP top 10 vulnerabilities while designing any services (frontend, backend, etc.)
o. Please follow SonarQube JavaScript, typescript, jsx, NodeJS, etc. design rules.
p. Please use `useMaterialReactTable` hook of material react table npm library when using material react table library.
q. Please define table's columns in a separate hook and import that hook where table component's useMaterialReactTable hook is being used. Also place these hooks related files under hooks/mrt folder.
r. Please prefix all of the Redis related keys with cw-expense-tracker: string.
s. Those components files which do not fall under any category shall be placed directly under components folder, i.e., components/. This rule is valid for all other components also such as hooks etc.
t. Please define axios instance under api folder.
u. Please define apis under different service files and put those all files under service folder. This rule is valid for frontend services.
v. Please define business logic functions under services folder. This rule is for backend services.
w. There will be two frontends namely – client and admin panel. Out of these two frontends client frontend (which will be used by normal users) must be fully responsive.
x. All frontend systems must follow W3C WCAG in all respects.
y. In addition to W3C WCAG standards, the admin panel must adhere to GIGW standards.
z. The backend services, bff services must clear all popular points which are raised during VAPT audits.
aa. Please apply rate limiting at backend and all frontend services must gracefully handle rate limiting errors sent by the backend.

bb. Please apply the error page by default in all of the frontend application.
cc. Please handle authentication, authorization, properly both at all backend and frontend applications/ services.
dd. Please apply the jsdoc of all react based components and helper functions.
ee. Please define all MUI dialog boxes under a separate component and place them under components/modal folder.
ff. For all add and delete operations, please use MUI dialog boxes.
gg. Before doing any create/ update/ delete/ logout operations, kindly ask a confirmation from the user. Please use predefined Confirmation Dialog box react component for this.
hh. After completing the application, you've to generate the SRS, SDD, documents and place them under docs/ folder.
ii. As there are links to Terms & Conditions and Privacy Policy in predefined Footer component, you've to design these components by yourself according to the application needs and place them under components/docs folder.

7. **Documentation of captcha service –** Please find the swagger json open api specification of the captcha service at http://localhost:11905/app/v1/docs/json or alternatively you can see the attached json file namely captcha-service.json.
8. **Predefined React.js Components –** Please note that in code, please put them under components/core folder.
    a. **OptionCard:** Please use this component when you need to make use of checkboxes. Please see the attached document markdown document namely Components.md for its complete code.
    b. **FileUploader:** Please use this component when you need to make use of file upload activities. Please see the attached document markdown document namely Components.md for its complete code.
    c. **Footer:** Please use this component for all frontend application's footer needs. Please see the attached document markdown document namely Components.md for its complete code.
    d. **ConfirmationDialog:** Please use this component for showing confirmation messages to the user. Please see the attached document namely Components.md for its complete code.
    e. **AppTheme:** Please use this component for frontend application's theme. Please see the attached document namely Components.md for its complete code.
    f. **PDFViewer:** Please use this component for showing any pdf blob received from the backend api on the frontend. Please see the attached document namely Components.md for its complete code.